SKILL

11 4 202

Reporting Workable Items

11 4 149

(§ . . . .

§ . . . .

§ . .)

11 4 171

A report that is only acknowledged is a lost requirement. Answering "yes, that's a bug, I'll look into it" in prose — without creating a tracked item — is a §11.4.197 requirements-integrity violation: the requirement was accepted, then silently evaporated. Every report becomes a **real, fully-populated, fully-synced workable item**, or it is explicitly and honestly refused.

The three report directives

Directive	Use when	Resulting Type
`/helix:bug` (`/default-bug`)	A product **defect** — regression, user-visible broken behaviour	`Bug`
`/task`	An internal **workstream** — refactor, doc, infra, gate, audit	`Task`
`/issue`	**Anything trackable**, type not yet decided	classified into `{Bug | Feature | Task}`

Note: the bare `/bug` collides with Claude Code's built-in — always use `/helix:bug` or `/default-bug` (see the `action-prefix-system` skill).

Classification is a closed set (§ . .)

`Bug` · `Feature` · `Task`. **There is no fourth type and no "Issue" type** — `/issue` is an *entry point*, not a type. Inventing a type is a §11.4.16 violation.

- Classify from the report's own content, stated as **FACT** (§11.4.6 — never a guess).
- If the content does not determine the type, **ASK** (§11.4.66 / §11.4.105) before creating the item.
- Only when running autonomously where asking is impossible (§11.4.101), default to the lowest-stakes `Task`, record the literal `Classification: defaulted-to-Task (§11.4.16 ambiguity default — reclassify)` in the description, and surface the reclassification command to the operator. **Never silently assert a type the report does not support.**

What a GOOD report contains

The item's description must be **comprehensive and human-readable** — a team member who was not in the room must understand it without reading code (§11.4.171). Structure it (§11.4.148 D2):

1. **WHAT** — subject + problem/goal in one self-contained sentence. Never a bare fragment like "Composes with" or "Critical" (§11.4.91 forbids those).
2. **SCOPE / MANIFEST** — the components, files, surfaces, or platforms touched.
3. **REPRODUCTION** — for a `Bug`: the exact sequence that reproduces it. If a working reproduction already exists, use **its exact sequence** — a deviating repro that never reaches the precondition proves nothing (§11.4.199).
4. **ACCEPTANCE** — the observable, machine-checkable condition that means done (the §11.4.108 runtime signature: what must be true on a clean deployment).
5. **EVIDENCE** — captured artefact paths that back the report (§11.4.5 / §11.4.69): log, recording, sink-probe, crash trace, screenshot.

A one-liner that names neither subject nor problem is refused by the summary generators (§11.4.91) — write the heading as the operator would read it.

What creating an item actually does

Reporting is not a note-to-self. The engine (`constitution/scripts/reporting/report_item.sh`) drives the full chain:

1. **Create** in the workable-items SQLite **single source of truth** with a stable, auto-incremented ticket id (§11.4.93 / §11.4.95 / §11.4.54), Status `Queued` , the chosen Type, and the structured description above.
2. **Regenerate every derived document from the DB** — Issues / Fixed / the summaries and their HTML / PDF / DOCX siblings (§11.4.12 / §11.4.53 / §11.4.65 / §11.4.106). The docs are outputs; the DB is the source.
3. **Push to every configured external tracker** (§11.4.148 D5).

Anti-bluff on every step (§ . / § . .)

- A tracker whose credentials or client are **absent** is **SKIPPED with an honest reason** (§11.4.10 / §11.4.3). A push is **NEVER faked**, and its absence is **NEVER hidden**.
- Report back, always: the **assigned ticket id**, the **chosen type**, the **doc-sync verdict**, and **each tracker's verdict with its captured-evidence path** (§11.4.5 / §11.4.69).
- Claiming "created + synced" without those verdicts is a PASS-bluff at the reporting layer.

After the item exists

- A `Bug` that is actionable now → investigate under **§11.4.102 systematic-debugging** (root cause BEFORE any fix; the Iron Law is "NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST"), and auto-activate it without waiting to be asked (§11.4.102(D)).
- Lifecycle from here — statuses, closure vocabulary, reopens, the testing diary — is the `workable-item-lifecycle` skill.